E7: Introduction to Computer Programming
Midterm 1A

Tuesday, September 30

Full Name

Student ID Number

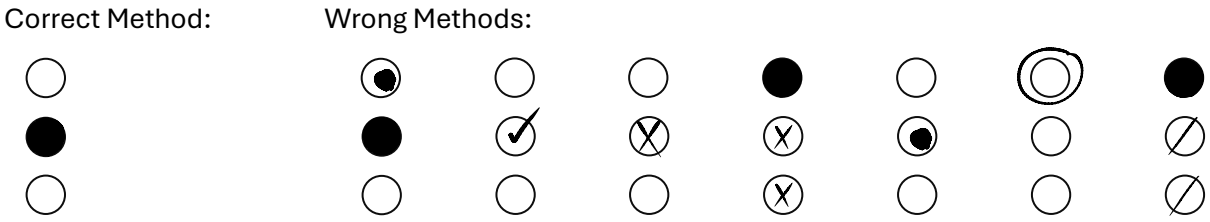
Instructions:

- You have 1 hour and 20 minutes to complete this exam.
- The exam is closed book, closed notes, closed computer, closed calculator.
- You may only use one 8.5" × 11" sheet of notes (both sides). No other resources are permitted.
- Mark your answers on the exam paper within the designated spaces. Responses written on scratch paper or outside of the specified answer spaces will not be graded.
- This exam is DOUBLE-SIDED. Make sure to read both sides of each page.
- The number of points associated with each part of each question is indicated.
- The exam is worth a total of 100 points.
- For questions with circular bubbles, you should select exactly *one* choice.
  - ◯ You may only select one option!
  - ◯ Or this one, but not both!
  - ◯ Or this one, but no more than one!
- For questions with square boxes, you may select *multiple* choices.
  - ☐ You can select this one!
  - ☐ You can select this one too!
  - ☐ You can select one, two, or all three of these!
- Only include your FINAL answers in the designated boxes. Any additional text will count as part of your answer and may result in a reduced grade.

Good luck!

Question 1 (30 points)

There are 10 multiple choice questions. There is only **ONE** correct choice for each question. Multiple bubbles, incomplete bubbles, or stray marks will be marked incorrect.



1. (3 pt) Which of the following is required in a Python conditional statement?

● `if`	○ `return`	○ `elif`
○ `else`	○ comparison operator	○ `elseif`

2. (3 pt) What is the final value of `out` after executing the following code?

```
out = 1
for a in range(1, 4):
    out *= a
```

● 6	○ 24	○ Error	○ 0
○ 2	○ 3	○ 4	○ 1

3. (3 pt) Consider the function below. The body of the function has been omitted on purpose. Which of the following calls passes different argument values than the others?

```
def fun(a, b, c=2):
    # function body and return statement omitted
```

○ `fun(1, c=2, b=3)`	○ `fun(1, 3)`	● All are equivalent
○ `fun(1, 3, 2)`	○ `fun(b=3, a=1)`	○ `fun(a=1, b=3)`

4. (3 pt) What is the output of the following code?

```
x = 7
y = 8
y = x
7 = x
print(x > y)
```

● Error	○ True	○ 7
○ None of the options	○ 8	○ False

5. (3 pt) Which of the following is necessary to ensure that a recursive function terminates?

◯ A counter variable	● A base case	◯ A `continue` statement
◯ An `end` keyword	◯ A `break` statement	◯ A `terminate` statement

6. (3 pt) What is the output of the following code?

```
nums = [2, 6, 4, 8, 10]
print(nums[2])
```

◯ 10	● 4	◯ 8
◯ 6	◯ None of the options	◯ 2

7. (3 pt) Assume that the data structure `t = (22, 3, 24, 25)` has already been defined. You want to change 3 to 23. Which line of code is the most appropriate approach to achieve this?

◯ `t = t + (0, 20, 0, 0)`	◯ `t[2] = 23`	● `t = (22, 23, 24, 25)`
◯ `t = t + 20`	◯ `t[1] = 23`	◯ `t[3] = 23`

8. (3 pt) What is the output of the following code?

```
msg = "Hello" # + "World"
print(msg)
```

◯ `Hello#World`	◯ `World`	◯ `Hello\#+World`	◯ `HelloWorld`
◯ `msg`	◯ `Hello World`	● `Hello`	◯ `Error`

9. (3 pt) What is the final value of `result` after executing the following code?

```
p = True
q = False
result = p or q and q
```

◯ `False`	◯ 2	◯ 5
◯ None of the options	● `True`	◯ `Error`

10. (3 pt) What is the final value of `depth` after executing the following code?

```
import numpy as np
magnitude = np.array([6.1, 5.8, 6.0, 7.2])
depth = np.array([8.1, 8.5, 9.0, 11.2])
depth[magnitude < 6] = 0
```

◯ `[8.1, 0.0, 0.0, 11.2]`	◯ `[8.1, 9.0, 11.2]`	◯ `[6.1, 0.0, 0.0, 7.2]`
● `[8.1, 0.0, 9.0, 11.2]`	◯ `[6.1, 0.0, 6.0, 7.2]`	◯ `[8.1, 11.2]`

Question 2 (10 points)

The function below computes the energy per unit distance gained by an object moving under a constant force, where `F` is the force, `d` is the distance, and `loss` is energy lost due to friction. **For all parts, assume that the following function has been defined.**

```
def netEnergy(F, d, loss=0):
    energy = F * d - loss
    return energy / d
```

What is the output of the following expressions? If an error occurs, write "Error". If nothing is displayed, write "Nothing". If the evaluation would run forever, write "Forever".

Only include your FINAL answers in the designated boxes. Any additional text will count as part of your answer and may result in a reduced grade. You can use the space outside the boxes for scratch work.

(a) (3 pt) `print(netEnergy(2, 10))`

```
2
```

(b) (3 pt) `print(netEnergy(2, 10, 10))`

```
1
```

(c) (2 pt) What is the output of the following code?

```
f = 2
d = 10
print(netEnergy(d, f))
```

```
10
```

(d) (2 pt) What is the output of the following code?

```
netEnergy(2, 10)
while energy < 5:
    print(netEnergy(2, 10))
```

```
Error
```

Question 3 (10 points)

For each part below, fill in the blank of the input code so that it produces the specified output exactly. You may only write one statement on each line. You may not use any functions or methods, built-in or otherwise. You may not modify the level of indentation of any line.

(a) (2 pt) Fill in blank (a).

```
    break
```

Input:

```
for k in [1, 2, 3, 4, 5]:
    if k == 3:
        ______(a)______
    print(k)
```

Output:

```
1
2
```

(b) (2 pt) Fill in blank (b).

```
    continue
```

Input:

```
for k in [1, 2, 3, 4, 5]:
    if k % 2 == 0:
        ______(b)______
    print(k)
```

Output:

```
1
3
5
```

(c) (2 pt) Fill in blank (c).

```
  return
OR
  return None
```

Input:

```
def mystery(n):
    for k in range(n):
        if k > 1:
            ______(c)______
        print(k)
    print("Done")

mystery(5)
```

Output:

```
0
1
```

(d) (2 pt) Fill in blank (d).

```
    k = 1
```

(e) (2 pt) Fill in blank (e).

```
while k < 5
OR
while k <= 4
```

Input:

```
______(d)______
while ______(e)______:
    k += 1
    print(k)
```

Output:

```
2
3
4
5
```

Question 4 (8 points)

Assume the code below has been executed. Notes:

- `list.append(x)`: Append item `x` to the end of the list
- `str.replace(old, new)`: Return a copy of the string with all occurrences of substring `old` replaced by `new`

```
fact = "PythonRocks123"
primes = [2, 3, 5, 7]
scores = (100, 90, 80, 70)
steps = range(0, 10, 2)

value = primes[-1] + steps[1]
fact.replace('123', '100')
primes.append(11)
new_scores = scores[:2] + (90, 100) * 2
```

What is the output of the following expressions? If an error occurs, write "Error". If nothing is displayed, write "Nothing".

(a) (2 pt) `print(fact)`

```
PythonRocks123
```

(b) (2 pt) `print(primes)`

```
[2, 3, 5, 7, 11]
```

(c) (2 pt) `print(value)`

```
9
```

(d) (2 pt) `print(new_scores)`

```
(100, 90, 90, 100, 90, 100)
```

Question 5 (10 points)

(8 pt) For each code block, assume the following variables have been defined: `one = 1`, `four = 4`, and `five = 5`. For each block, select what will be displayed after execution. Select ALL that apply.

(i)
```
if one > four:
    print('a')
elif one + four == five:
    print('b')
elif one < four:
    print('c')
else:
    print('d')
print('e')
```

☐ d ☐ Nothing ☐ a
■ b ■ e ☐ c

(iii)
```
if one + four == five:
    print('a')
    if one > four:
        print('b')
        if four < five:
            print('c')
    else:
        print('d')
print('e')
```

☐ c ☐ b ☐ Nothing
■ a ■ e ■ d

(ii)
```
if one > four:
    print('a')
if one + four == five:
    print('b')
if one < four:
    print('c')
else:
    print('d')
    print('e')
```

☐ Nothing ■ c ☐ d
☐ a ☐ e ■ b

(iv)
```
if one > four:
    print('a')
    if one + four == five:
        print('b')
    elif one + four > five:
        print('c')
    print('d')
elif one + four < five:
    print('e')
```

☐ b ☐ d ■ Nothing
☐ a ☐ e ☐ c

(2 pt) For the code block below, also assume the same variables have been defined: `one = 1`, `four = 4`, and `five = 5`. Which of the following options will be displayed after execution? Note that `...` means the same output repeats indefinitely.

(v)
```
while one < five:
    if one == four:
        print('a')
        break
    if one + four > five:
        print('b')
        continue
    print('c')
    one += 1
print(one)
```

○
```
c
b
b
a
4
```

○
```
c
b
b
a
```

●
```
c
b
b
b
...
```

○
```
c
b
2
```

○
```
c
...
```

○
```
c
b
b
5
```

○ Error

○ Nothing

Question 6 (15 points)

You need to write a Python function `terrain(p)` that takes a list of numbers (`int` or `float`) and determines if it contains exactly one distinct peak. A peak is defined as follows:

- It is strictly greater than its immediate neighbors (elements immediately to the right and left).
- It cannot be the first or last element.
- There must be exactly one such peak in the list.

Your function should return an integer representing the index of the peak or `-999` if the list does not have exactly one valid peak. Refer to the examples below for more details. Anything written on this page will NOT be graded.

```
def terrain(p):
    """Check if p has exactly one peak; return peak index or -999
    >>> terrain([0, 5, 2]) # 5 is the peak
    1
    >>> terrain([0, 5, 7, 9, 8]) # 9 is the peak
    3
    >>> terrain([10, 7, 9, 8]) # 9 is the peak (peak cannot be at the boundary)
    2
    >>> terrain([0, 8, 7, 9, 6]) # 2 peaks (8 and 9)
    -999
    >>> terrain([0, 5, 7]) # peak cannot be at the boundary
    -999
    >>> terrain([0, 9, 9, 9, 8]) # peak not distinct
    -999
    """
    peak = False # track if a peak has been detected
    index = -999
    for i in ______(a)______:
        if ______(b)______:
            if ______(c)______:
                ______(d)______
            index = ______(e)______
            ______(f)______
    return index
```

You will implement the function `terrain` by filling in the blanks. You may only write one statement on each line. You may not use ; to place two statements on the same line. Except when explicitly provided in multiple-choice questions, you may not use any functions or methods, built-in or otherwise. You may not modify the level of indentation of any line.

(a) (3 pt) Which option correctly fills blank (a)?

○ `range(1, len(p) + 1)`	○ `range(1, len(p) - 2)`	○ `p`
○ `range(len(p) - 1)`	● `range(1, len(p) - 1)`	○ `range(len(p))`
○ `range(len(p) + 1)`	○ `range(1, len(p))`	○ `enumerate(p)`

(b) (4 pt) Fill in blank (b).

```
if p[i] > p[i-1] and p[i] > p[i+1]
```

(c) (2 pt) Fill in blank (c).

```
if peak # OR peak == True
```

(d) (2 pt) Fill in blank (d).

```
return -999
```

(e) (2 pt) Fill in blank (e).

```
index = i
```

(f) (2 pt) Fill in blank (f).

```
peak = True
```

Question 7 (15 points)

An arithmetic sequence is a sequence of numbers in which each term is obtained by adding a constant common difference d to the previous term. If the sequence starts at a, it can be written as: $a,\ a+d,\ a+2d,\ a+3d,\ ...,\ a+(n-1)d$, where n is the number of terms. For example, the sequence $1, 4, 7, 10$ has $a = 1$, $d = 3$, and $n = 4$.

Consider two Python functions `sum_iter` and `sum_rec` that each take three scalar arguments: `a` (integer representing the first term), `d` (integer representing the common difference), and `n` (positive integer representing the number of terms). Both functions should return the sum of the `n` terms of the arithmetic sequence. `sum_iter` should use iteration, whereas `sum_rec` should use recursion. Any solutions that do not use iteration or recursion to compute the sum will receive 0 points.

```
def sum_iter(a, d, n):
    """
    >>> sum_iter(1, 3, 2)
    5 # 1 + 4 = 5
    >>> sum_iter(1, 3, 4)
    22 # 1 + 4 + 7 + 10 = 22
    >>> sum_iter(2, 3, 1)
    2 # only 1 term
    """
    total = 0

    for i in ______(a)______:

        ______(b)______

______(c)______
```

```
def sum_rec(a, d, n):
    """
    >>> sum_rec(1, 3, 2)
    5 # 1 + 4 = 5
    >>> sum_rec(1, 3, 4)
    22 # 1 + 4 + 7 + 10 = 22
    >>> sum_rec(2, 3, 1)
    2 # only 1 term
    """
    if ______(d)______:

        ______(e)______

    else:

        return ______(f)______
```

The parts for this question begin on the next page. You can use the space below for scratch work. Anything written on this page will NOT be graded. Only answers within the designated boxes on the next page will be graded.

You will implement both functions by filling in the blanks. You may not use ; to place two statements on the same line. You may not use any functions, other than the recursive function inside `sum_rec` and those provided in multiple choice options. Do not modify the level of indentation of any line.

(a) (2 pt) Which option correctly fills blank (a)?

- ○ `n`
- ○ `range(len(n+1))`
- ● `range(n)`
- ○ `range(len(n))`
- ○ `range(n+1)`
- ○ `range(len(n-1))`
- ○ `len(n)`
- ○ `range(n-1)`

(b) (4 pt) Fill in blank (b).

```
total += a + i * d # OR total = total + a + i * d
```

(c) (4 pt) Which option correctly fills blank (c)?

○
```
def sum_iter(a, d, n):
    total = 0
    for i in ______(a)______:
        ______(b)______
return total
```

○
```
def sum_iter(a, d, n):
    total = 0
    for i in ______(a)______:
        ______(b)______
        return total
```

●
```
def sum_iter(a, d, n):
    total = 0
    for i in ______(a)______:
        ______(b)______
    return total
```

○
```
def sum_iter(a, d, n):
    total = 0
    for i in ______(a)______:
        ______(b)______
    print(total)
```

(d) (2 pt) Fill in blank (d).

```
if n == 1 # OR n == 0
```

(e) (2 pt) Fill in blank (e).

```
return a # OR return 0 if (d) is n == 0
```

(f) (1 pt) Fill in blank (f). **This part is only worth 1 point, and some might find it challenging. Finish the rest of the exam first!**

```
return a + sum_rec(a + d, d, n - 1) # OR a + (n-1) * d + sum_rec(a, d, n - 1)
```

Question 8 (2 points)

The parts below are unrelated; treat each part as a separate question. **Both parts are only worth 2 points, and some might find them challenging. Finish the rest of the exam first!**

(a) (1 pt) Consider the function below, which takes a single scalar float as input and returns its ceiling. The ceiling of a number is the smallest integer greater than or equal to that number. For example, the ceiling of 4.0 is 4, the ceiling of 4.01 is 5, and the ceiling of 4.99 is 5.

```
def ceil_value(value): return ______(a)______
```

Fill in blank (a) to return the ceiling of `value`. You may only use the built-in `int` function. No other functions are allowed. You may not use conditional statements (`if/else`). No partial credit will be awarded for incorrect solutions.

```
return int(value) + (value > int(value))
# The second set of () is required
```

(b) (1 pt) Consider the function below, which takes three scalar integers as input, such that `m` and `n` are distinct values and `p` is guaranteed to be equal to either `m` or `n`. The function should return the value of `m` or `n` that is different from `p`.

```
def other_value(m, n, p): return ______(b)______
>>> other_value(1, 2, 1)
2
>>> other_value(4, 3, 4)
3
```

Fill in blank (b) to return the desired value. You may not use any built-in functions. You may NOT use `return m + n - p` (or any trivial variation of it). You must provide two distinct solutions. "Distinct" means that the underlying method is different. Simply reversing or rephrasing the same idea will not count as distinct. No partial credit will be awarded for incorrect solutions.

Solution 1:

```
return (p == m) * n + (p == n) * m # OR (p != m) * m + (p != n) * n
```

Solution 2:

```
return m if p == n else n # OR n if p == m else m
```